Intelligent Adaptive Battery charger Using ESP32 Microcontroller

A report submitted in fulfilment of the requirements for the award of the completion of

Summer Internship

By

S .Venkatesan

(23BEC0368)



ELECTRICAL AND ELECTRONIC ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY

TIRUCHIRAPALLI-620015

June 2025

Intelligent Adaptive Battery charger Using ESP32 Microcontroller

A report submitted in fulfilment of the requirements for the award of the completion of

Summer Internship

By

S. Venkatesan

(23BEC0368)



ELECTRICAL AND ELECTRONIC ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY

TIRUCHIRAPALLI-620015

June 2025

BONAFIDE CERTIFICATE

This is to certify that the project titled **"Intelligent Adaptive Battery charger Using ESP32 Microcontroller**" is a Bonafide record of the work done by

S. Venkatesan (23BEC0368)

VELLORE INSTITUTE OF TECHNOLOGY

VELLORE

in fulfillment of the requirements for the award of the completion **of Summer Internship Program** in **Specialization** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI,** during 19/05/2025 - 21/06/2025.

Dr. S. Moorthi

Professor/ EEE

(Guide)

Abstract

In modern battery-powered applications, charging safety, efficiency, and adaptability have become critical. This project focuses on the design and implementation of a microcontroller-based smart battery charger tailored for AA lithium-ion cells. The system intelligently transitions through four charging phases—trickle, constant current (CC), pulse charging, and constant voltage (CV)—based on real-time voltage, current, and temperature feedback. MATLAB Simulink was used for initial system modelling and validation using PID control, ensuring stability and responsiveness of the charging profile.

An ESP32 microcontroller was employed to execute the control logic in hardware, utilizing analog sensors and a PWM-driven MOSFET to regulate charge delivery. The charger responds dynamically to battery conditions, switching modes to optimize charge time, battery life, and thermal safety. Key features include temperature cut-off, automatic mode shifting, and real-time signal monitoring. This project demonstrates a complete embedded solution that combines simulation, firmware development, and circuit integration to create a flexible and robust charging system for lithium-ion batteries.

Keywords:

Acknowledgment

I express my sincere gratitude to the Department of Electrical and Electronics Engineering, National Institute of Technology, Tiruchirappalli, for providing me the opportunity to undertake this research internship and supporting my learning experience with valuable resources and guidance.

I am deeply thankful to my project supervisor, **S. Moorthi**, for his invaluable mentorship, technical insights, and consistent encouragement throughout the project. His guidance was instrumental in shaping the outcome of this work.

I would also like to thank the faculty members **Mr. Venkatesan** and **Mrs. Somaiya** for their technical advice and support during hardware testing and simulation validation. I'm grateful to the laboratory staff for their assistance and cooperation throughout the implementation phase.

A heartfelt thanks to my teammate **Hemann**, whose dedicated help, brainstorming sessions, and hands-on support made this project journey both productive and enjoyable. I also extend appreciation to my peers and fellow interns for their positive spirit and collaborative contributions.

Lastly, I thank my family and friends for their constant encouragement, and my home institution, **Vellore Institute of Technology (VIT)**, for equipping me with the foundational knowledge and motivation to pursue this research endeavour.

TABLE OF CONTENTS

Title **Page No.**

ABSTRACT i
ACKNOWLEDGMENTS ii
TABLEOF CONTENTS ..... …iii

Chapter 1 – Introduction

1.1 Project Overview 6
1.2 Objectives 6

Chapter 2 – Methodology

2.1 System Architecture 7
2.2 Component Description 7
2.3 Block Diagram 8

Chapter 3 – MATLAB Simulation

3.1 Model Overview 9
3.2 Operating Logic 10
3.3 Observations and Analysis 11
3.4 PWM Signal Generation 11

Chapter 4 – Code Implementation

4.1 Charging State Transitions 13
4.2 Code Highlights 13

Chapter 5 – Hardware Implementation

5.1 ESP32 Description 14
5.2 Sensor Integration 14
5.3 Power Regulation Stage 14
5.4 Battery Configuration 15
5.5 Safety and Protections 15
5.6 Hardware Assembly and Testing 15
5.7. Final Hardware Integration 16

Chapter 6 – Results and Discussion

6.1 Observed Behaviour 18
6.2 Sensor Accuracy and Response 18
6.3 PWM Signal Analysis 18
6.4 System Efficiency and Safety 19
6.5 Comparison with Simulation 19

Chapter 7 – Conclusion 20

Chapter 8 – References 21

Chapter-1

1. Introduction

1.1. Project Overview

This project focuses on the development of a fast and intelligent charging system tailored for lithium-ion AA batteries. Leveraging the capabilities of the ESP32 microcontroller, the charger dynamically operates across four controlled modes—Trickle, Constant Current (CC), Pulse, and Constant Voltage (CV). These modes are automatically selected based on continuous feedback from voltage, current, and temperature sensors to ensure efficient energy delivery and battery safety.

To validate the control logic before hardware implementation, a MATLAB Simulink model was developed using PID controllers. The simulation helped fine-tune charging parameters and ensure stable system behaviour. The final prototype integrates these findings with real-time PWM regulation through a MOSFET driver circuit, enabling responsive and thermally safe fast charging.

This embedded system provides an adaptable and practical solution for managing lithium-ion charging in real-world use cases, combining simulation-driven design, sensor-based monitoring, and microcontroller-based control.

1.2. Objectives

- To design and implement a charger for AA lithium-ion batteries with four charging modes
- To use MATLAB simulation for validating control strategies
- To implement real-time control using ESP32 and sensor inputs
- To regulate charging via PWM and MOSFET driver circuitry

Chapter 2

Methodology

2.1 System Architecture

The smart battery charger system is structured into four key layers:

- Sensing Layer: Captures battery voltage, current, and temperature using a voltage divider, current sensor, and LM35 temperature sensor. These analog signals are read by the ESP32's ADC pins.
- Control Layer: An ESP32 microcontroller runs a state machine with four modes—Trickle, CC, Pulse, and CV—using PID controllers to regulate output based on sensor feedback.
- Power Regulation Layer: The ESP32 generates a PWM signal to drive a MOSFET through a gate driver, controlling the charging current. Passive filters smooth the output to the battery.
- Protection Layer: The system halts charging if temperature exceeds 70 °C or if the battery reaches full voltage with minimal current. Real-time data is also logged via serial output for monitoring.

2.2.Component Description

ESP32: Central controller, generates PWM, reads sensors

LM35: Analog temperature sensor (10 mV/°C)

Voltage Divider: Scales battery voltage to ADC range

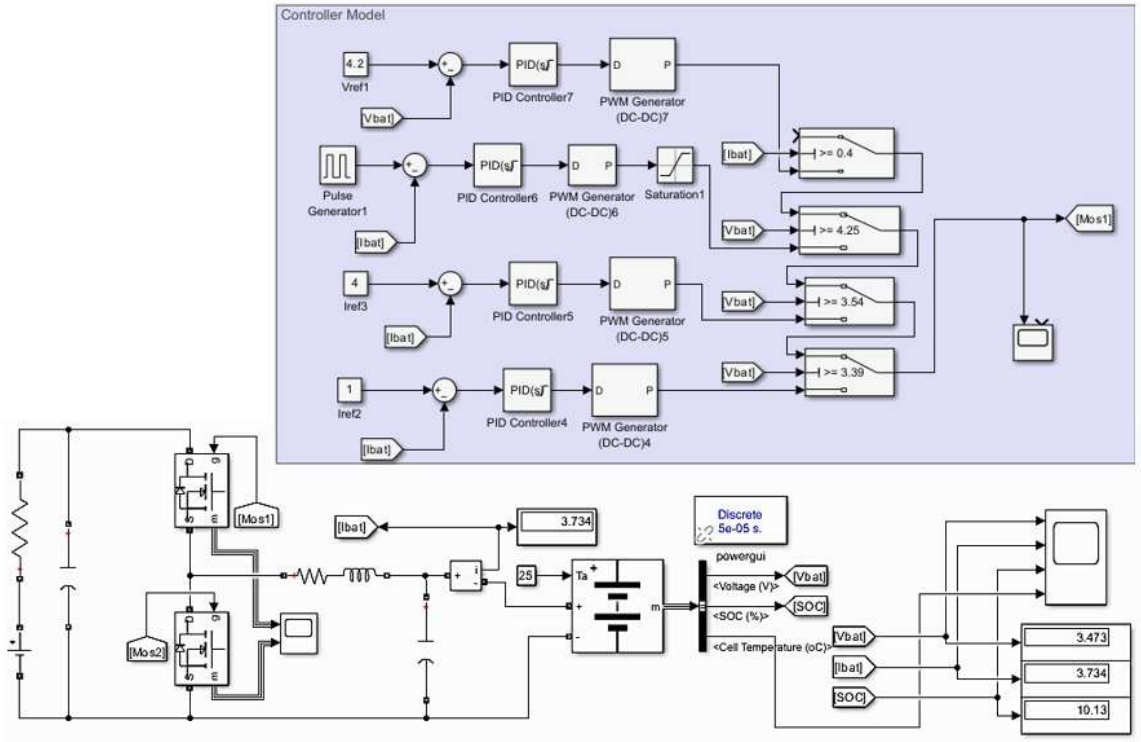
Current Sensor: Converts sensed current to voltage

MOSFET + Driver: Power stage for current regulation

Battery: Single-cell lithium-ion AA (3.2–4.2 V range)

PID Library: Arduino PID_v1 for PWM regulation

2.3.Block diagram :



Chapter-3

MATLAB Simulation

To validate the charging strategy before hardware implementation, a MATLAB Simulink model of the smart battery charger was developed. The simulation aimed to replicate the real-time behaviour of a lithium-ion charging system with four modes: trickle, constant current (CC), pulse, and constant voltage (CV).

3.1 Model Overview

The Simulink model incorporates key building blocks to represent control, switching, sensing, and battery behaviour:

Battery Model: Emulates the behaviour of a lithium-ion AA cell, providing real-time voltage (Vbat), current (Ibat), and temperature (Ta) feedback.

PID Controllers: Three separate PID controllers were used to regulate different aspects of the charging process:

One for current control during trickle and CC modes.

One for voltage control during CV mode.

One for pulse modulation during pulse charging.

MOSFET Switches (Mos1, Mos2): Modelled using ideal switch blocks, driven by PWM signals, to control current flow based on the active mode.

PWM and Pulse Generators: Created the required switching waveforms for MOSFETs. PWM duty cycle was dynamically adjusted based on PID controller output.

SOC Calculation Block: Computed the State of Charge (SOC) based on current integration, allowing monitoring of charge progression.

Temperature Sensor Model: Simulated battery temperature response to charging current for over-temperature protection logic.

Display and XY Graphs: Visualized real-time plots of voltage, current, temperature, and SOC for analysis.

3.2 Operating Logic

The simulation was organized as a sequential finite-state system:

Trickle Mode: Low current applied with PID regulation until Vbat ≥ 3.2 V.

CC Mode: Current set to a high value (e.g., 3 A), maintaining constant flow until Vbat ≥ 3.7 V.

Pulse Mode: Switched periodically using a pulse generator to deliver 0.5 A pulses at 1 second intervals.

CV Mode: PID controlled the duty cycle to maintain Vbat at 4.2 V until Ibat ≤ 0.05 A.

Safety Control: Simulation halted charging if battery temperature exceeded a defined threshold (e.g., 70 °C).

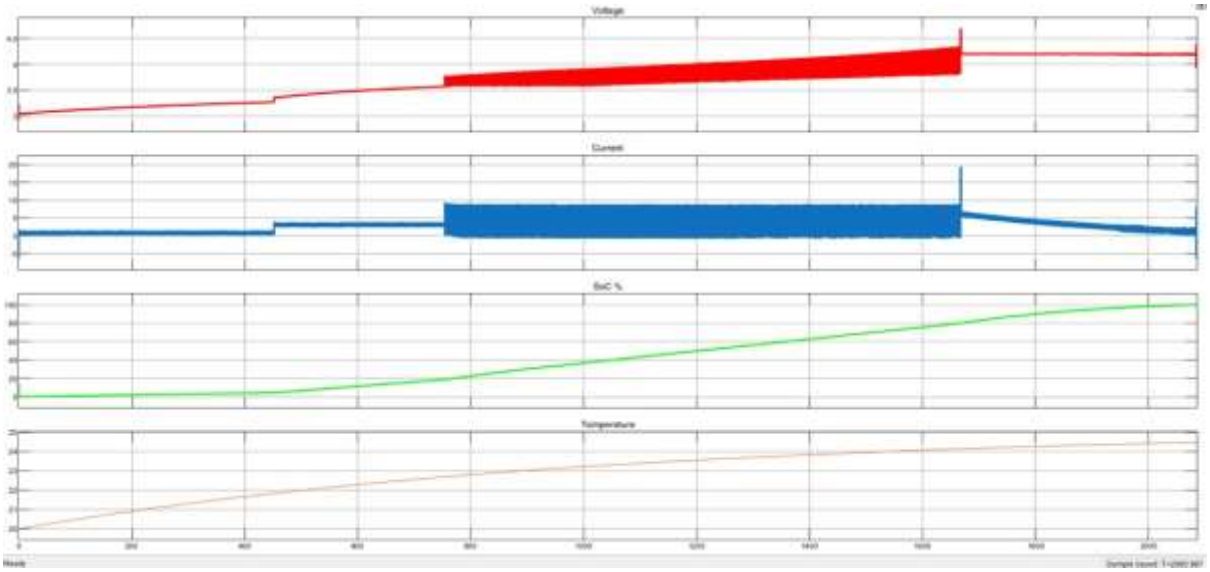
3.3 Observations

The battery voltage and current profiles showed smooth transitions between modes.

SOC increased steadily and saturated during CV mode.

PWM signals were responsive to PID control and matched expected behaviour.

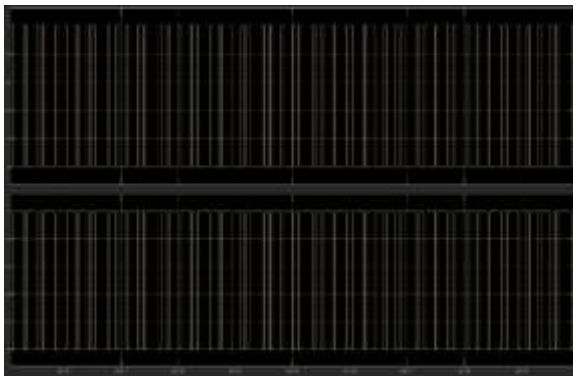
Over-temperature protection and automatic cutoff were successfully simulated.



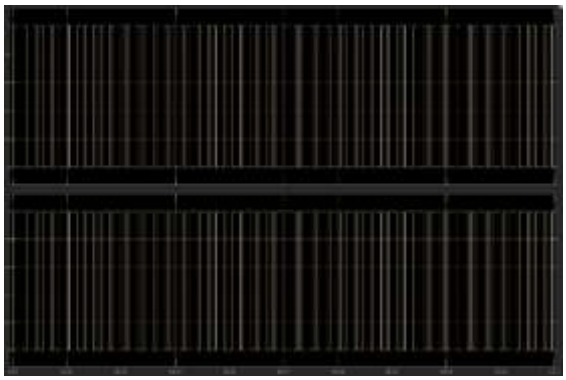
3.4 . PWM Signals generation

The following plots illustrate the PWM signals generated in MATLAB Simulink for each charging mode:

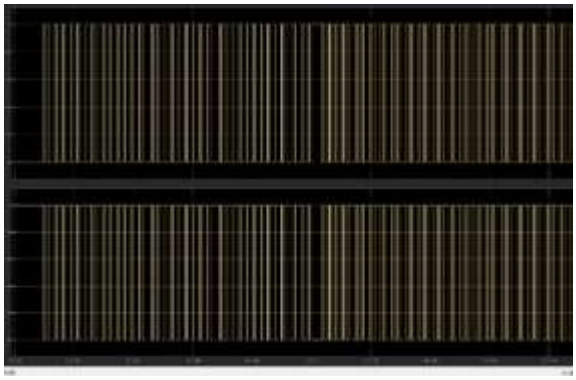
- **Trickle Mode**: Shows a relatively low and stable duty cycle, ensuring a gentle initial current flow to revive deeply discharged cells.
- **Constant Current (CC) Mode**: Exhibits a higher and consistent duty cycle to maintain a fast and steady charge current as the battery voltage rises.
- **Pulse Mode**: Displays a square-wave pattern with on-off cycles at defined intervals (e.g., 1 second), enabling mid-stage fast charging while minimizing heat buildup.
- **Constant Voltage (CV) Mode**: Demonstrates a gradually decreasing duty cycle as the battery approaches its maximum voltage, allowing current to taper down safely.



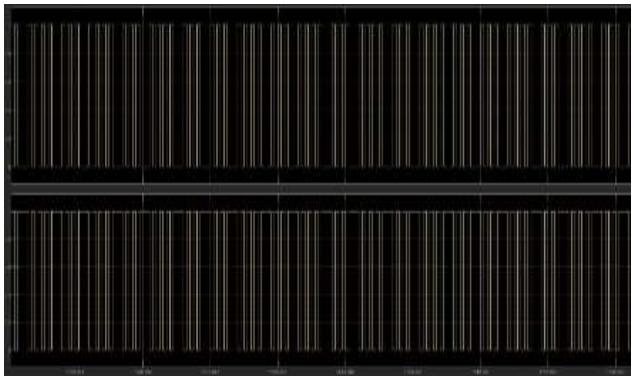
Trickle mode



constant current



Pulse charging



constant voltage

These plots confirm the correct functioning of the PWM control strategy and highlight the system's ability to modulate charging power in real time. The changes in duty cycle directly reflect the responsiveness of the PID controllers under different battery conditions.

Chapter-4
Code Implementation

The ESP32 firmware was implemented using the Arduino IDE. Each charging mode is implemented as a state in a state machine. Voltage, current, and temperature are continuously monitored using the ADC.

4.1.Charging States:

TRICKLE: Delivers ~0.6 A until Vbat ≥ 3.2 V

CC: Maintains 3.0 A until Vbat ≥ 3.7 V

PULSE: Alternates 0.5 A pulses every second until Vbat ≥ 4.2 V

CV: Maintains 4.2 V until Ibat ≤ 0.05 A

DONE: PWM = 0; charging complete or overtemp condition

4.2.Code Highlights:

Two PID controllers: ccPID (current control), cvPID (voltage control)

Sensor calibration done using voltage scaling for ESP32 ADC

PWM generated on GPIO25 using ledcWrite() with 5 kHz frequency

Safety: charging stopped if temperature ≥ 70 °C

Chapter -5

Hard ware implementation

5.1.esp32 description

ESP32: Served as the central controller for all operations. Its dual-core architecture, integrated ADC channels, and PWM capabilities made it suitable for high-speed real-time control.

PWM Output: Generated using ledcWrite() at 5 kHz on GPIO25 to drive a power MOSFET via a gate driver.

State Machine: Implemented in firmware to control the transition between Trickle, CC, Pulse, CV, and Done modes.

5.2.Sensor Integration

LM35 Temperature Sensor: Monitored battery surface temperature. The analog voltage (10 mV/°C) was fed into ESP32's ADC and converted to °C in software. A cutoff was triggered at 70 °C.

Voltage Sensing: A voltage divider network scaled battery voltage to within ESP32's ADC range (0–3.3 V).

Current Sensing: Used a shunt resistor and differential amplifier (or hall-effect current sensor) to measure charging current, also routed to ADC input. Calibration ensured accuracy.

5.3.Power Regulation Stage

MOSFET Driver: Interfaced between ESP32's logic-level PWM output and the power-stage MOSFET, ensuring fast switching with minimal losses.

Power MOSFET: Controlled current flow into the lithium-ion battery. Selected based on low R_DS(on) and high current handling.

Filtering Circuit: Passive components (capacitors and inductors) were used to smooth the PWM output and reduce ripple in the battery current.

5.4. Battery Configuration

Lithium-ion AA Cell: Single-cell rechargeable battery with a typical operating range between 3.2 V and 4.2 V. Selected for its availability and safe nominal current levels during testing.

5.5. Safety and Protections

Over-temperature protection was implemented in code using the LM35 readings.

Charging was stopped automatically when:

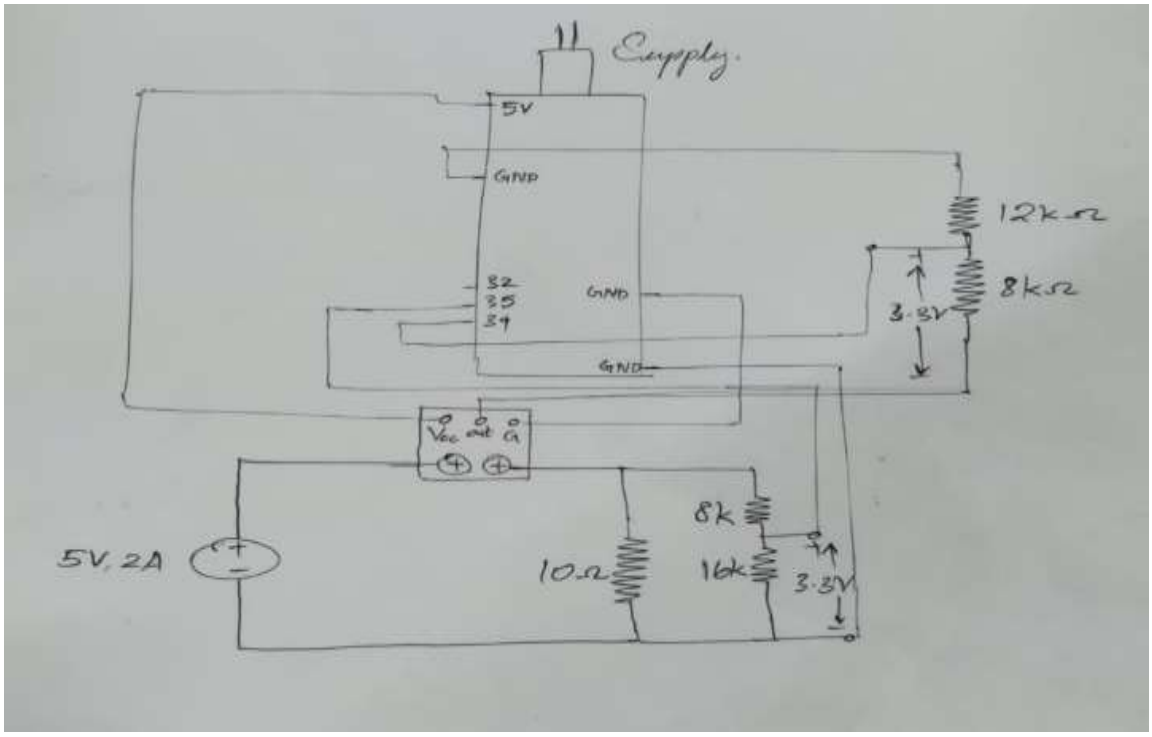
Temperature exceeded 50 °C

Voltage reached 4.2 V and current dropped below 0.05 A

Any sensor returned abnormal or zero readings

5.6 .Testing and Validation Setup

Circuit Setup for Code and Sensor Testing



Oscilloscope (DSO) captures of PWM output



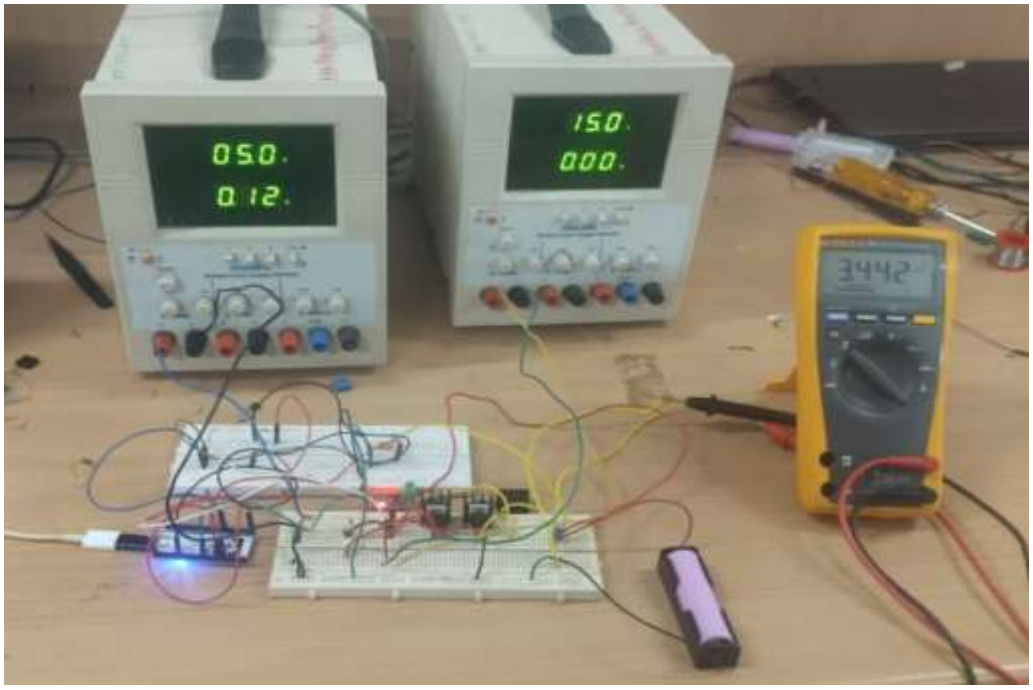


Trickle **cc**



Pulse charging

5.7. Final Hardware Integration



The ESP32 was centrally placed with sensors and the MOSFET circuit arranged around it for easy wiring. The LM35 was attached to the battery, and voltage/current sensors were positioned close to the terminals. A single ground reference was maintained, and the PWM output was routed cleanly to the MOSFET driver. Color-coded wiring minimized confusion during testing.

Google Drive link to the video demo

https://drive.google.com/file/d/1V7cRoPvnt-9VjYbihG7zm7UMLuTiPfQJ/view?usp=drivesdk

https://drive.google.com/file/d/1VKY9iHNeztuImZ45b2WtIEPsjOdGDe0-/view?usp=drivesdk

Chapter -6
Results and Discussion

The smart battery charger was successfully implemented and tested under controlled conditions using a single lithium-ion AA cell. Key performance parameters such as charging current, battery voltage, temperature, PWM output, and state transitions were recorded throughout the process.

6.1. Observed Behaviour

- **Trickle Mode**: The charger initially supplied ~0.6 A until the battery voltage crossed 3.2 V. The PID controller kept the current stable despite minor voltage fluctuations, confirming accuracy in low-current control.
- **Constant Current (CC) Mode**: The charger increased the current to ~3 A. The ESP32 maintained steady PWM duty via PID control, and the battery voltage rose smoothly to 3.7 V without overshoot, enabling faster energy input.
- **Pulse Mode**: The system alternated between ON and OFF cycles at 1-second intervals, delivering ~0.5 A during pulse highs. This mode helped dissipate heat between bursts while accelerating mid-stage charge acceptance.
- **Constant Voltage (CV) Mode**: Once the battery reached 4.2 V, the charger reduced current output and held the voltage constant. The PID voltage controller smoothly adjusted PWM to taper down current, as expected.
- **Done Mode**: When the charging current fell below 0.05 A or if temperature exceeded 70 °C (as tested in simulation), the charger shut off PWM entirely. Charging completion was clearly observable both in hardware and serial output.

6.2. Sensor Accuracy and Response

- The voltage and current sensing circuits offered reliable feedback within the ADC resolution of the ESP32.
- Temperature readings from the LM35 were linear and responsive, allowing the software to enforce thermal protection logic promptly.

6.3. PWM Signal Analysis

- The PWM output waveform was observed using a Digital Storage Oscilloscope (DSO).
- Duty cycle varied dynamically in response to feedback, closely following the output of the respective PID controller.

- No jitter or noise artifacts were observed that could harm MOSFET operation or battery stability.

6.4. System Efficiency and Safety

- The system charged the battery from ~3 V to 4.2 V in a significantly shorter time than conventional CC-CV-only chargers by front-loading higher current in early stages.
- Temperature remained well within the safety threshold throughout due to adaptive mode switching and pulse cooling intervals.
- No unexpected cutoffs or overvoltage behaviour were detected during testing, confirming the reliability of the control logic.

6.5. Comparison with Simulation

- Hardware behaviour closely matched the MATLAB Simulink model, especially in mode transitions, voltage profile, and charge rate.
- Minor discrepancies (e.g., ADC quantization effects, hardware noise) were within acceptable limits and did not affect overall system performance.
- The validation proved that simulation-based tuning of PID controllers was effective and translated well into real-world results.

Chapter 7

Conclusion

This project successfully demonstrated the design and implementation of a fast and intelligent lithium-ion battery charger using an ESP32 microcontroller. By leveraging real-time voltage, current, and temperature feedback, the system dynamically managed charging across four modes—Trickle, Constant Current (CC), Pulse, and Constant Voltage (CV)—prioritizing both speed and safety. The integration of PID-based control logic, PWM-driven power regulation, and sensor-driven decision-making resulted in stable and efficient battery charging behaviour.

The simulation of charging algorithms in MATLAB Simulink ensured proper tuning of control loops and validated performance before physical prototyping. Hardware testing further confirmed accurate mode transitions, responsive PWM modulation, and reliable sensor feedback. Overall, the system met its objective of safe fast charging while preserving battery health.

Future Enhancements

To improve the system's scalability and practicality, the following enhancements are proposed:

- **AC to DC Conversion and Relay Integration**: Adding an AC-DC input stage with relay control would allow the charger to be powered directly from mains supply and enable load-side isolation.
- **PCB Design**: Moving the design from breadboard to a printed circuit board would increase durability, reduce wiring complexity, and improve electrical stability.
- **Multi-Cell Battery Management**: Expanding the firmware and hardware architecture to support simultaneous charging and balancing of multiple cells would enhance usability for larger battery packs.

These future upgrades would transform the prototype into a more robust, real-world ready charging platform suited for embedded and consumer applications.

Chapter -8

References

S. Kumar and R. Singh, "Design of Smart Charger for Series Lithium-Ion Batteries," *International Journal of Power Electronics and Drive Systems*, vol. 12, no. 3, pp. 215–222, 2023.

A. Sharma, M. Prakash, and V. Mehta, "Battery Charger with a Capacitor-Diode Clamped LLC Resonant Converter," *IEEE Transactions on Power Electronics*, vol. 38, no. 1, pp. 112–120, Jan. 2023.

Y. Zhang and H. Li, "Design and Simulation of Lithium-Ion Battery Charger Using Forward Power Converter for Hybrid Energy Storage System," in *Proc. IEEE Energy Conversion Congress and Exposition (ECCE)*, Detroit, MI, USA, Oct. 2022, pp. 1456–1463.

R. B. Patel and K. Ghosh, "Adaptive Fast-Charging Techniques for Lithium-Ion Batteries Using PWM-Controlled Boost Topologies," *IEEE Access*, vol. 10, pp. 56234–56244, 2022.

L. Chen, J. Wang, and T. Yao, "Modelling and Implementation of Smart Battery Charging with Mode Switching Control," *IEEE Journal of Emerging and Selected Topics in Power Electronics*, vol. 9, no. 4, pp. 4292–4300, Aug. 2021.

Code:

```
#include <PID_v1.h>

// PID control variables
double ccSetpoint, ccInput, ccOutput;
double cvSetpoint, cvInput, cvOutput;

PID ccPID(&ccInput, &ccOutput, &ccSetpoint, 2.0, 5.0, 1.0, DIRECT);
PID cvPID(&cvInput, &cvOutput, &cvSetpoint, 2.0, 5.0, 1.0, DIRECT);

enum ChargingState {
  TRICKLE,
  CC,
  PULSE,
  CV,
  DONE
};

ChargingState chargingState = TRICKLE;

const int pwmPin = 25;
const int currentPin = 34;
const int voltagePin = 35;
const int tempPin = 32;

unsigned long pulseStartTime = 0;
bool pulseHigh = true;

const float trickleCurrent = 1.0;
const float ccCurrent = 4.0;
const float pulseCurrent = 0.5;
const float cvVoltage = 4.2;
const float doneCurrent = 0.05;

float readCurrent() {
  long sum = 0;
  for (int i = 0; i < 100; i++) {
    sum += analogRead(currentPin);
    delayMicroseconds(100);
  }
  int adc = sum / 100;
  float voltage = adc * (3.3 / 4095.0);
  float offsetVoltage = 3.3 / 2.357;
  float current = ((voltage - offsetVoltage) / 0.185) - 0.19;
  return current;
}

float readVoltage() {
  int adc = analogRead(voltagePin);
  float raw = (adc * (5.0 / 4095.0)) + 0.1;
  return raw;
}

int controlCharging(float vbat, float ibat) {
  int pwmValue = 0;

  if (ibat >= 0.4 && vbat >= 4.25) {
    // CV mode (PID7)
    cvSetpoint = cvVoltage;
    cvInput = vbat;
    cvPID.Compute();
    pwmValue = cvOutput;
    chargingState = CV;
    if (ibat <= doneCurrent) chargingState = DONE;
```

```
  } else if (vbat >= 3.54) {
    // PULSE mode (PID6) - 100% duty with 10kHz switching
    chargingState = PULSE;
    if (millis() - pulseStartTime > 1000) {
      pulseHigh = !pulseHigh;
      pulseStartTime = millis();
    }
    if (pulseHigh) {
      ledcSetup(0, 10000, 8);  // 10kHz frequency
      ledcAttachPin(pwmPin, 0);
      pwmValue = 255;
    } else {
      pwmValue = 0;
    }

  } else if (vbat >= 3.39) {
    // Constant Current mode (PID5) - 50% duty
    chargingState = CC;
    pwmValue = 128;

  } else {
    // Trickle mode (PID4) - 25% duty
    chargingState = TRICKLE;
    pwmValue = 64;
  }

  if (chargingState == DONE) {
    pwmValue = 0;
  }

  return constrain(pwmValue, 0, 255);
}

void setup() {
  Serial.begin(115200);
  pinMode(pwmPin, OUTPUT);
  ledcSetup(0, 5000, 8);         // Default: 5kHz, 8-bit
  ledcAttachPin(pwmPin, 0);     // PWM channel 0

  ccPID.SetMode(AUTOMATIC);
  ccPID.SetOutputLimits(0, 255);
  cvPID.SetMode(AUTOMATIC);
  cvPID.SetOutputLimits(0, 255);

  Serial.println("Battery Charging Controller Started");
}

void loop() {
  float vbat = readVoltage();
  float ibat = readCurrent();

  int pwmValue = controlCharging(vbat, ibat);
  ledcWrite(0, pwmValue);

  Serial.print("Vbat: "); Serial.print(vbat); Serial.print(" V, ");
  Serial.print("Ibat: "); Serial.print(ibat); Serial.print(" A, ");
  Serial.print("PWM: "); Serial.print(pwmValue); Serial.print(", ");
  Serial.print("State: ");

  switch (chargingState) {
    case TRICKLE: Serial.println("Trickle (25% duty)"); break;
    case CC:      Serial.println("Constant Current (50% duty)"); break;
    case PULSE:   Serial.println("Pulse (100% duty @10kHz)"); break;
    case CV:      Serial.println("Constant Voltage (PID-controlled)"); break;
    case DONE:    Serial.println("Done"); break;
  }
```